

VEGA SENTINEL

AI-Powered Predictive Anomaly Detection at the Edge

Edge-AI Multi-Sensor Anomaly Detection for Enclosed Space Safety

Team Abnormal Detectors

EMBRIX'26 – VEGATHON 2026

Track 2 – Edge AI & TinyML Challenge

Sri Sivasubramaniya Nadar College of Engineering (SSNCE)

September 2026

1 Team Members

Name	Role / Contribution
Hariharan S	Project Lead
Sriarthan R	Hardware Lead
Maheshwaran T	Software Lead
Tamilarasan S	Testing

2 Abstract

VEGA SENTINEL is an AI-powered predictive anomaly detection system that analyses real-time multi-sensor data entirely at the edge. The prototype combines a VEGA ARIES v2.0 RISC-V controller with three environmental sensors (MQ-2 gas/smoke sensor, DHT22 temperature/humidity sensor, and an IR flame sensor) and a buzzer alert mechanism. A sliding-window feature pipeline feeds a trained logistic-regression model that executes directly on the microcontroller in C++, so no cloud communication, data transfer, or network round-trip is required for a safety response.

Project-specific measured results:

- Trained model (Model A) executed on the ARIES v2.0 board: reference normal window scored $p = 0.018949 \rightarrow \text{NORMAL}$, reference abnormal window $p = 0.992752 \rightarrow \text{ABNORMAL}$ (both correct).
- 7-fold stratified cross-validation accuracy on the collected dataset: **0.9446** (development result; random window split, leakage caveats apply).
- Leakage-aware leave-one-trial-out validation: **0.7449** — the honest development figure.
- Real controlled flame trial: flame sensor detected the flame; one live window scored $p \approx 0.34 < 0.5$ (missed detection) — documented honestly; dataset expansion and retraining are in progress. Real-world accuracy is *not yet validated*.

3 Introduction

Enclosed spaces — laboratories, server rooms, storage areas, workshops and hostel rooms — can develop dangerous conditions from smoke, gas-related events, abnormal temperature/humidity drift, or fire. Conventional threshold alarms examine one sensor against a fixed limit and ignore how conditions change *together over time*, while cloud-based smart systems depend on connectivity, add latency and cost, and stop working when the network drops — unacceptable for a safety function.

VEGA SENTINEL brings predictive anomaly detection closer to the monitored space itself: sensor information is collected, cleaned, analysed by a lightweight edge model, classified, and used to trigger an immediate local alert. The decision function is *learned from collected data*, not a set of hand-tuned thresholds, and runs entirely on the indigenous VEGA RISC-V platform.

4 Problem Statement

Predictive Analytics & Anomaly Detection. The project aims to detect abnormal conditions from multi-sensor temporal patterns and provide early warning of possible hazardous conditions in enclosed spaces, using Edge AI / TinyML on a low-cost edge device (VEGA ARIES v2.0), with no dependence on cloud connectivity.

5 Objectives

1. Monitor relevant environmental conditions in real time (temperature, humidity, gas/smoke-related sensor response, flame presence).
2. Preprocess sensor readings with a validity gate and sliding windows to reduce noise and obtain useful temporal features.
3. Use a lightweight edge-based AI/TinyML approach (trained logistic-regression classifier executing in C++) for pattern analysis.
4. Classify operating conditions as Normal or Abnormal (data collection additionally used a Warning label).
5. Analyse changing temporal patterns (window dynamics and multi-window persistence) to support early indication of a developing abnormal condition.
6. Trigger an audible alert so preventive action can be taken, with temporal confirmation to suppress single-window noise.

6 Proposed System

VEGA SENTINEL combines a VEGA ARIES v2.0 controller with multiple sensors and an alert mechanism. The prototype components are an IR flame sensor, a DHT22 temperature and humidity sensor, an MQ-2 gas sensor, and a buzzer driven through a 2N2222 transistor stage. The overall concept is to process sensor information at the edge and generate an immediate local response when abnormal behaviour is detected.

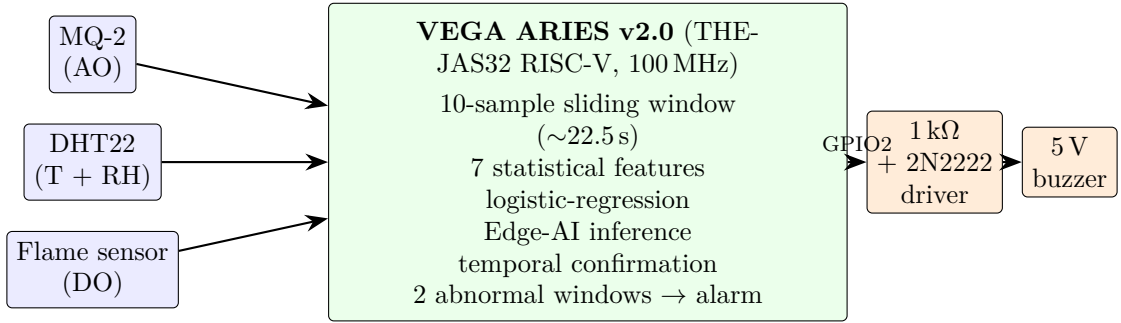
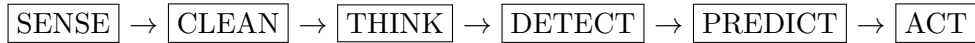


Figure 1: Proposed system block diagram.

7 System Architecture

The workflow follows the pattern:



SENSE Collect live sensor data every 2500 ms: DHT22 (bit-banged 1-wire driver, checksum-verified), MQ-2 analog output through a $2 \times 2.2\text{ k}\Omega$ divider into A0 (raw ADC counts), flame sensor digital output (active-LOW) on GPIO1.

CLEAN Validate every sample before use: DHT22 checksum and plausibility range checks. Invalid samples are discarded; the sliding window is *not* shifted and the alarm state is preserved. Valid samples update a 10-sample window ($\sim 22.5\text{ s}$ of history) from which 7 statistical features are computed.

THINK A lightweight trained model (logistic regression with embedded standardisation constants, implemented in plain C++) converts the 7 features into an abnormality probability on the microcontroller itself.

DETECT Each window is classified NORMAL or ABNORMAL against a fixed 0.5 threshold (never tuned to force a demonstration).

PREDICT Temporal confirmation analyses the *sequence* of AI decisions: 2 consecutive abnormal windows indicate a developing condition and arm the alarm; 3 consecutive normal windows are required to clear it.

ACT GPIO2 switches the buzzer through a 1 k Ω base resistor and a 2N2222 transistor from an external 5 V rail, so the 12 mA-limited GPIO never drives the buzzer directly.

8 Hardware Components

S.No	Component	Qty	Key specification / role
1	VEGA ARIES v2.0 board	1	THEJAS32/VEGA ET1031 RISC-V @ 100 MHz, 256 KB SRAM, 2 MB flash, 3.3 V I/O, 4 analog inputs (ADS1015)
2	IR Flame Sensor module	1	3-pin, 3.3 V, digital DO, active-LOW
3	DHT22 sensor	1	Temperature + humidity, 3.3 V, 1-wire protocol
4	MQ-2 gas sensor module	1	5 V, analog AO output (gas/smoke-related response)
5	Buzzer (5 V active)	1	Audible alarm, external 5 V supply
6	2N2222 transistor	1	Buzzer switching driver
7	Resistor 1 k Ω	1	2N2222 base current limit
8	Resistor 2.2 k Ω	2	MQ-2 analog voltage divider

[INSERT IMAGE HERE]

Upload `photo_components.pdf` or `photo_components.png`
to this Overleaf project, or replace this box
with `\includegraphics`.

9 Function of Components

- **VEGA ARIES v2.0:** main edge-computing/controller board; performs acquisition, windowing, feature extraction, AI inference, confirmation logic and alarm control.
- **IR Flame Sensor:** senses the presence of flame (experimentally verified active-LOW output).
- **DHT22:** measures temperature and relative humidity.
- **MQ-2 Gas Sensor:** gas/smoke-related sensing; its raw analog response is used as an uncalibrated sensor signal (no ppm claims).
- **Buzzer + 2N2222 + 1 k Ω :** audible alert stage; the transistor driver keeps the buzzer current off the 12 mA-limited GPIO.
- **2 $\times$ 2.2 k Ω divider:** halves the MQ-2 module's 5 V analog output so it stays within the safe input range of the ARIES analog input.

10 Sensor Data Acquisition

The prototype uses the following sensor inputs, sampled on a common cadence:

- IR flame detection (binary, active-LOW).
- Temperature and humidity from DHT22 (checksum-verified, bit-banged driver).
- Gas/smoke-related raw analog reading from MQ-2 via the divider on A0.
- **Sampling interval: 2500 ms** (all sensors; bounded by the DHT22 requirement of ≥ 2 s between reads).

Sensor operating ranges / calibration details:

- DHT22: 0–100 %RH, -40 to $+80$ °C (observed in sessions: 27.8 – 28.8 °C and 68.7 – 80.3 %RH).
- MQ-2: **raw ADC counts 0–2047** on this board (onboard ADS1015, single-ended, measured effective range; observed clean-air baseline ~ 101 – 259 counts). The MQ-2 is *not* a calibrated gas-concentration instrument; values are raw sensor responses, not ppm.
- Flame sensor: digital, active-LOW (LOW = flame detected), verified experimentally on our module.

11 Data Preprocessing

The system filters noisy or inconsistent sensor readings and converts raw readings into useful inputs for the edge model.

Actual filtering method: every acquisition is read into temporary variables first and validated (DHT22 checksum + plausibility range, MQ-2 ADC range). Only a *valid* sample updates the sliding window. On failure the sample is discarded, the window is not shifted, no inference runs, and the alarm state is preserved — stale values are never used as new measurements. (This fixed an earlier firmware issue where the window was shifted before validation.)

Features calculated (over a 10-sample window, ~ 22.5 s):

Feature	Definition
humidity_mean	mean relative humidity in window
temp_mean	mean temperature in window
mq2_range	max – min of MQ-2 raw ADC in window
mq2_delta	last – first MQ-2 value in window (trend)
mq2_std	sample std ($n - 1$) of MQ-2 in window
humidity_std	sample std ($n - 1$) of humidity
flame_fraction	fraction of window samples with flame detected

Table 1: The 7 model input features.

The same window/feature logic is used offline for training-data generation and on-device at runtime, so the model’s learned feature distribution matches deployment.

12 Edge AI / TinyML Implementation

13 Anomaly Detection and Prediction

The system classifies the operating condition and examines changing patterns to estimate a possibly developing abnormal condition.

Prediction criteria / failure indicators: the window dynamics (mq2_range, mq2_delta, mq2_std, humidity_std) capture a condition that is *developing* across ~ 22.5 s, and the multi-window persistence counters require the condition to persist across ~ 5 s of consecutive abnormal decisions — providing early warning of a developing hazard rather than reacting to a single noisy sample.

Model / algorithm	Logistic regression (sigmoid probability) with embedded standardisation (Model A). Trained, not hand-tuned thresholds. Inference is direct C/C++ arithmetic on the board; no TensorFlow Lite Micro / Edge Impulse (not verified on ARIES v2.0).
Dataset	5 collection sessions (~ 45 min total, 1085 valid samples) $\rightarrow$ 107 windows: 100 normal / 7 abnormal . A window is abnormal if it contains any critical event label.
Input features	7 (table above); fixed order enforced across training script, exported header, and firmware.
Output classes	2 deployed: NORMAL / ABNORMAL (data collection additionally used normal/warning/critical labels).
Training platform	Python 3 + scikit-learn (<code>StandardScaler</code> + <code>LogisticRegression</code> , <code>class_weight="balanced"</code> , <code>random_state=42</code>); constants exported automatically to a C++ header.
Accuracy	7-fold stratified CV: 0.9446 (development estimate; random window split, overlap leakage possible). Leave-one-trial-out: 0.7449 . On-board reference windows: $p = 0.018949 \rightarrow$ NORMAL, $p = 0.992752 \rightarrow$ ABNORMAL (math execution verified on ARIES). Real-world accuracy <i>not yet validated</i> .
Model size	22 constants (7 means, 7 scales, 7 coefficients, intercept) $\approx$ 88 bytes of flash; the entire inference routine including feature code is well under 1 KB.
Inference time	Not yet formally benchmarked. The computation is 7 multiply-adds and one sigmoid per window step ($\ll 1$ ms at 100 MHz, estimated).

Table 2: Edge-AI implementation summary (all values measured or reproducible from the repository; nothing fabricated).

14 Alert Mechanism

When an abnormal condition is confirmed, the system triggers an immediate local alert.

- **Buzzer activation condition:** 2 consecutive AI windows classified ABNORMAL ($p \geq 0.5$). The alarm clears after 3 consecutive normal windows.
- **Other alert mechanism:** the firmware streams a CSV telemetry line and an explicit AI decision line (`AI,p=...,status=NORMAL/ABNORMAL`) over USB serial at 115200 baud for live monitoring and dataset logging.

15 Circuit and Wiring

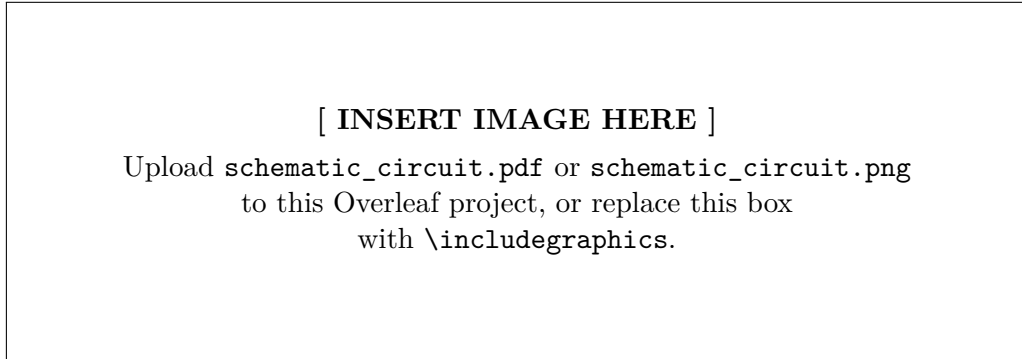
Design rules encoded in the circuit: (1) no GPIO ever drives the buzzer directly — the 12 mA per-I/O limit forbids it, so the 2N2222 stage does the switching; (2) the 5 V MQ-2 output never reaches an analog pin directly — the divider keeps A0 at or below 2.5 V; (3) a common ground across the board, all sensors and the buzzer supply is mandatory.

State	Detection condition	System response
Normal	Window probability $p < 0.5$	No alarm; buzzer OFF; CSV logging continues.
Warning	Mild deviation label used during data collection (not a deployed classifier class); a single abnormal window in isolation	Logged for review; deliberately no alarm yet — the temporal confirmation layer suppresses single-window noise.
Anomaly	$p \geq 0.5$ on 2 consecutive windows	Buzzer ON via 2N2222 driver; cleared only after 3 consecutive normal windows .

Table 3: Detection states and responses.

Component	VCC	GND	Signal / output pin
DHT22	ARIES 3.3 V	ARIES GND	DATA $\rightarrow$ GPIO0 (module pull-up to 3.3 V; the VEGA core provides no internal pull-up)
IR Flame Sensor	ARIES 3.3 V	ARIES GND	DO $\rightarrow$ GPIO1 (active-LOW: LOW = flame detected)
MQ-2 Gas Sensor	5 V	GND (common)	AO $\rightarrow$ $2 \times 2.2\text{ k}\Omega$ divider $\rightarrow$ A0 (halves the 5 V output; DO unused)
Buzzer	External 5 V (+)	via driver to common GND	GPIO2 $\rightarrow$ $1\text{ k}\Omega \rightarrow$ 2N2222 base; emitter $\rightarrow$ GND; collector $\rightarrow$ buzzer (−)

Table 4: Verified wiring summary.



16 Software Implementation

The software continuously acquires sensor information, validates it, performs the required preprocessing, analyses the resulting features, determines the current condition, and activates the alert mechanism when required.

17 Algorithm / Program Flow

1. Initialise the VEGA ARIES v2.0 board: serial (115200 baud), GPIO0 (DHT22), GPIO1 (flame, input), GPIO2 (buzzer, output, OFF), A0 (MQ-2).

2. Every 2500 ms, read all sensors into temporary variables.
3. Validate the DHT22 sample (checksum + plausibility). If invalid: discard the sample, keep the window and alarm state unchanged, and wait for the next cycle.
4. On a valid sample: shift the 10-sample window and insert the newest values.
5. When the window is full: compute the 7 features and standardise them with the embedded scaler constants.
6. Run the logistic-regression model (dot product + sigmoid) and classify the window NORMAL ($p < 0.5$) or ABNORMAL ($p \geq 0.5$).
7. Update the temporal confirmation counters: 2 consecutive abnormal windows arm the alarm; 3 consecutive normal windows clear it.
8. Drive the buzzer accordingly through the 2N2222 stage and emit the CSV/decision telemetry lines.
9. Repeat the monitoring cycle.

18 Code

The complete, tested source code is in the project repository:

<https://github.com/hariharanboss/VEGA-Sentinel>

(firmware under `firmware/`, reproducible ML pipeline under `ml/`, datasets under `dataset/`). The core on-device inference and confirmation logic, verbatim from the final firmware:

```
// Model A inference: standardise -> linear score -> sigmoid
float modelPredict(float features[])
{
    float score = MODEL_INTERCEPT;
    for (int i = 0; i < MODEL_NUM_FEATURES; i++)
    {
        float standardized =
            (features[i] - MODEL_MEANS[i]) / MODEL_SCALES[i];
        score += MODEL_COEFFS[i] * standardized;
    }
    if (score > 30.0f) return 1.0f; // numerically safe
    if (score < -30.0f) return 0.0f; // sigmoid clamps
    return 1.0f / (1.0f + expf(-score));
}

// Temporal confirmation: 2 consecutive abnormal -> alarm ON,
// 3 consecutive normal -> alarm OFF (buzzer via 2N2222)
void processWindowClassification(bool abnormal)
{
    if (abnormal) {
        consecutiveAbnormal++; consecutiveNormal = 0;
        if (consecutiveAbnormal >= 2 && !alarmActive) {
            alarmActive = true; } // buzzer ON
    } else {
        consecutiveNormal++; consecutiveAbnormal = 0;
        if (consecutiveNormal >= 3 && alarmActive) {
            alarmActive = false; } // buzzer OFF
    }
    digitalWrite(PIN_BUZZER, alarmActive ? HIGH : LOW);
}
```

19 Required Libraries

- **Firmware:** no external libraries. The VEGA Arduino core (C-DAC v1.1.2) provides the Arduino API used (`Serial`, `pinMode`, `digitalRead`, `analogRead`, `micros`); the DHT22 driver is self-contained bit-banged code and the model is plain C/C++ arithmetic.
- **PC pipeline (training/data tools):** `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `pyserial`.

20 Code Explanation

Initialization: serial at 115200 baud; GPIO0 and GPIO1 configured; GPIO2 set as output, LOW (buzzer off); A0 analog input.

Sensor reading: DHT22 via the bit-banged one-wire driver with checksum verification (≥ 2 s between reads); MQ-2 as a single `analogRead` (raw counts); flame via `digitalRead` with active-LOW normalisation.

Data preprocessing: validity gate before any state change; fixed-size 10-sample window (static arrays, no dynamic allocation); 7 features with sample standard deviation ($n - 1$), matching the offline training computation exactly.

AI/TinyML inference: embedded scaler constants (mean/scale per feature) + logistic-regression coefficients + sigmoid, all generated automatically from the training pipeline into `model.h`.

Classification: per-window binary decision against the fixed 0.5 threshold.

Prediction: multi-window persistence analysis (consecutive abnormal/normal counters) that turns per-window decisions into a stable early-warning state.

Buzzer control: GPIO2 drives only the 2N2222 base through 1 k Ω (≈ 2.6 mA); the buzzer itself is switched from the external 5 V rail.

21 Testing and Results

Testing verified sensor readings, classification behaviour, and alert response under normal and selected *safe*, *supervised* abnormal conditions.

Test	Condition / expected	Actual result	Remarks
1	Board bring-up: serial prints at 115200	Pass	IDE + VEGA core (ARIES v2) verified
2	Onboard ADC (ADS1015) stability	Pass	Stable counts; effective single-ended range 0–2047
3	DHT22 bit-banged read	Pass	Temperature/humidity track room; checksum errors rare
4	MQ-2 via divider	Pass	Clean-air baseline ~ 101 –259 counts; responds to safe stimulus; warm-up noted
5	Flame sensor polarity	Pass	Active-LOW verified; 5 flame-positive rows recorded in committed trials
6	Buzzer driver (2N2222)	Pass	ON/OFF on GPIO2; GPIO load ≈ 2.6 mA (within 12 mA limit)
7	Model A math on ARIES	Pass	$p = 0.018949 \rightarrow \text{NORMAL}$; $p = 0.992752 \rightarrow \text{ABNORMAL}$ (matches PC)
8	Live sliding-window inference	Pass	New AI decision every 2.5 s after window fill; DHT-failure window preservation verified
9	Controlled flame trial (live)	Partial	Flame detected by sensor; one live window scored $p \approx 0.34 \rightarrow \text{NORMAL}$, buzzer OFF

Table 5: Test record (all results actually executed; nothing fabricated).

Measured sensor values: temperature 27.8–28.8°C, humidity 68.7–80.3%RH, MQ-2 raw ADC 101–259 counts, flame binary (active-LOW), across 5 committed sessions on 2026-09-11.

Model quality (development, not final): random-split CV 0.9446; leave-one-trial-out 0.7449; a real flame trial produced one missed detection (test 9) — root cause is the small dataset (7 abnormal windows), not the architecture. The response is more data and retraining with trial-level validation, *not* a hard-coded flame-to-buzzer rule and *not* a tuned threshold.

Overall observed result: the end-to-end pipeline (sensors → windowing → on-board AI → confirmation → alarm) works on hardware. Final real-world performance is not yet validated; dataset expansion is in progress.

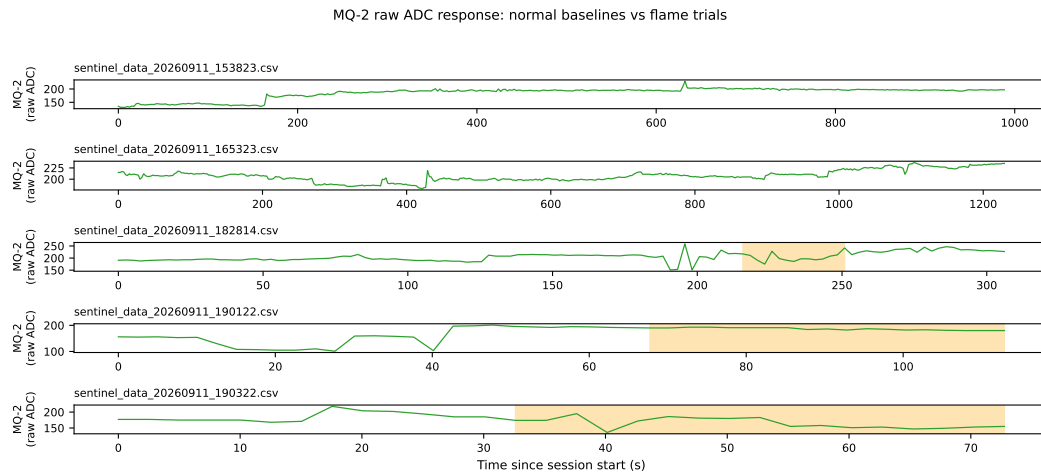


Figure 3: MQ-2 sensor response comparison across sessions.

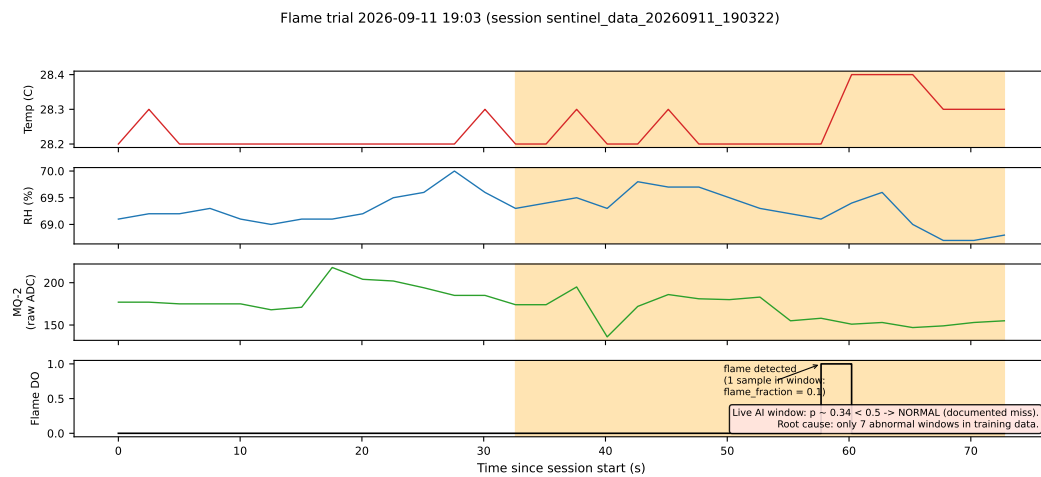


Figure 4: Flame-trial incident analysis.

22 Project Output

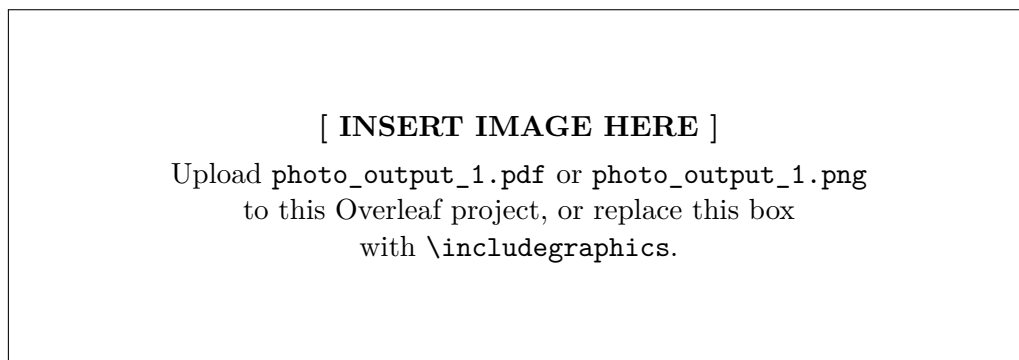


Figure 5: VEGA SENTINEL output / test photograph 1.

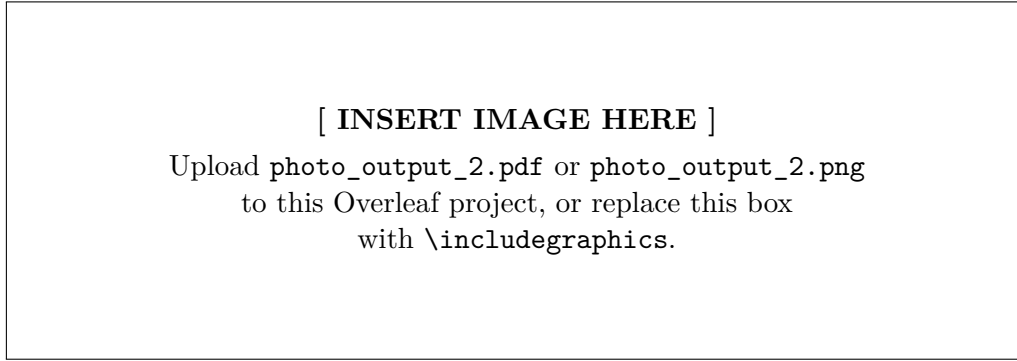


Figure 6: VEGA SENTINEL output / test photograph 2.

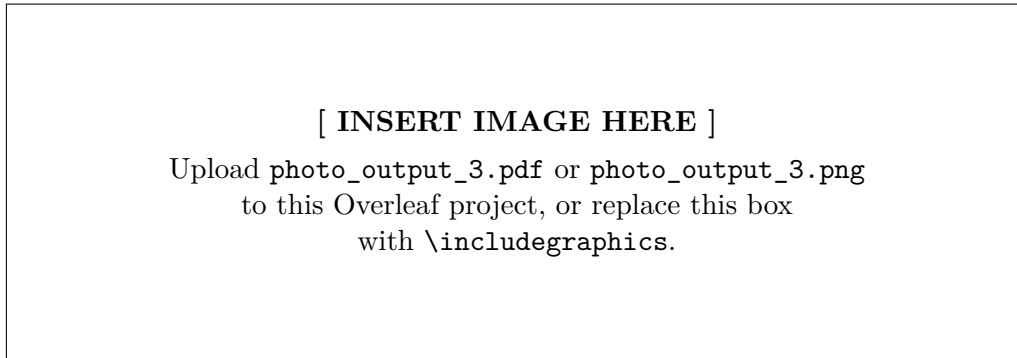


Figure 7: VEGA SENTINEL output / test photograph 3.

23 Feasibility and Scalability

Feasibility. The system uses low-cost sensors and a low-cost indigenous controller, making it affordable and easy to deploy. The model is deliberately tiny (88 bytes of constants, no framework), so it operates comfortably within the 256 KB SRAM / 2 MB flash of the ARIES v2.0, and every result in this report was produced on the actual hardware.

Challenges. Noisy or inconsistent sensor readings (DHT22 checksum failures, MQ-2 warm-up and baseline drift); different rooms produce different normal environmental patterns; small datasets weaken generalization (demonstrated honestly by test 9).

Proposed solutions. A validity gate before window updates (implemented); deployment-consistent window/feature generation (implemented); continuous data collection and trial-level validation to improve and honestly measure accuracy over time (in progress).

Scalability. The architecture extends from one enclosed space to many using the same edge-AI pattern (each unit learns its own room’s normal profile); additional sensors and anomaly types can be added by extending the feature set and retraining, without redesigning the system.

24 Impact and Benefits

- **Early detection:** abnormal temporal patterns are flagged while a condition is developing, enabling timely action.
- **Cost reduction:** low-cost hardware, no cloud bills, no connectivity requirements.
- **Improved safety:** local, latency-bounded warning for enclosed spaces (laboratories, server rooms, storage areas).
- **Energy efficiency:** edge processing removes continuous cloud communication.
- **Indian ecosystem:** a complete Edge-AI safety system demonstrated on the indigenous VEGA RISC-V platform, suitable for institutions and smart infrastructure.

25 Applications

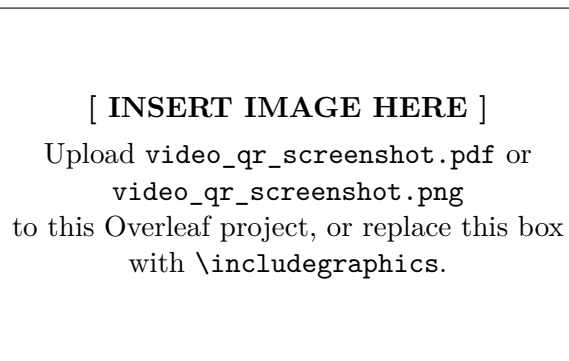
- Laboratory and workshop environmental safety monitoring.
- Server-room / storage-area abnormal-condition warning.
- Hostel and residential enclosed-space safety assistance.
- Baseline template for predictive maintenance on other low-cost edge nodes (same pipeline, different sensors).

26 Future Scope

- Dataset expansion: 20–30 minutes of fresh normal data plus multiple safe, supervised abnormal trials with recovery phases.
- Retraining with trial-level (leakage-aware) validation and deployment of the improved model via the automated export tool.
- On-device benchmarking of inference time and memory usage.
- Adaptive baselines for slowly drifting environments (humidity/MQ-2 warm-up).
- Enclosure/prototype packaging and long-duration soak tests.
- Optional long-term logging dashboards on the PC side.

27 Project Demonstration

Project video link: _____ (to be added)



28 Conclusion

VEGA SENTINEL presents an edge-based approach to predictive anomaly detection by combining multi-sensor monitoring with lightweight AI/TinyML processing on the indigenous VEGA ARIES v2.0 RISC-V platform. Its SENSE–CLEAN–THINK–DETECT–PREDICT–ACT workflow provides early warnings, removes dependence on cloud processing, and supports safer monitoring of enclosed spaces.

Final project results and performance: the complete pipeline is physically implemented and working — trained model executing on-board with math verified against the PC implementation ($p = 0.018949$ / $p = 0.992752$ on reference windows), live sliding-window inference every 2.5 s, and a hardware-verified alarm stage. Model quality is a *development result* (CV 0.9446 with leakage caveats; 0.7449 trial-level), a real flame-trial miss is documented, and final accuracy claims await the in-progress dataset expansion and retraining. The system is an honest, reproducible prototype — not a certified safety device, and no unmeasured performance is claimed.

29 CAD / PCB Design (Optional)

No custom PCB has been designed yet; the prototype uses the ARIES v2.0 development board with point-to-point wiring (see the Circuit and Wiring section). A dedicated PCB is future work.

30 References

1. A. Malhotra *et al.*, “A Holistic Review of the TinyML Stack for Predictive Maintenance,” *IEEE Access*, vol. 12, pp. 184861–184882, Dec. 2024. DOI: 10.1109/ACCESS.2024.3512860.
2. “TinyMLEdge: A Workflow for Deploying TinyML Models in Industrial Edge Devices,” 2024 IEEE 22nd Int. Conf. on Industrial Informatics (INDIN), 2024. DOI: 10.1109/INDIN58382.2024.10774465.
3. IEEE — TinyML & Predictive Maintenance: <https://ieeexplore.ieee.org/document/10781407>
4. IEEE — TinyMLEdge & Industrial Edge Devices: <https://ieeexplore.ieee.org/document/10774465>
5. IEEE — Anomaly Detection & Predictive Maintenance: <https://ieeexplore.ieee.org/document/10586631>
6. C-DAC, VEGA ARIES v2.0 product page: https://www.cdac.in/index.aspx?id=product_details&productId=ARIESv2.0; datasheet: https://vegaprocessors.in/files/ARIESv2%200_Datasheet_v2.pdf
7. F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.
8. Project repository (code, datasets, results): <https://github.com/hariharanboss/VEGA-Sentinel>

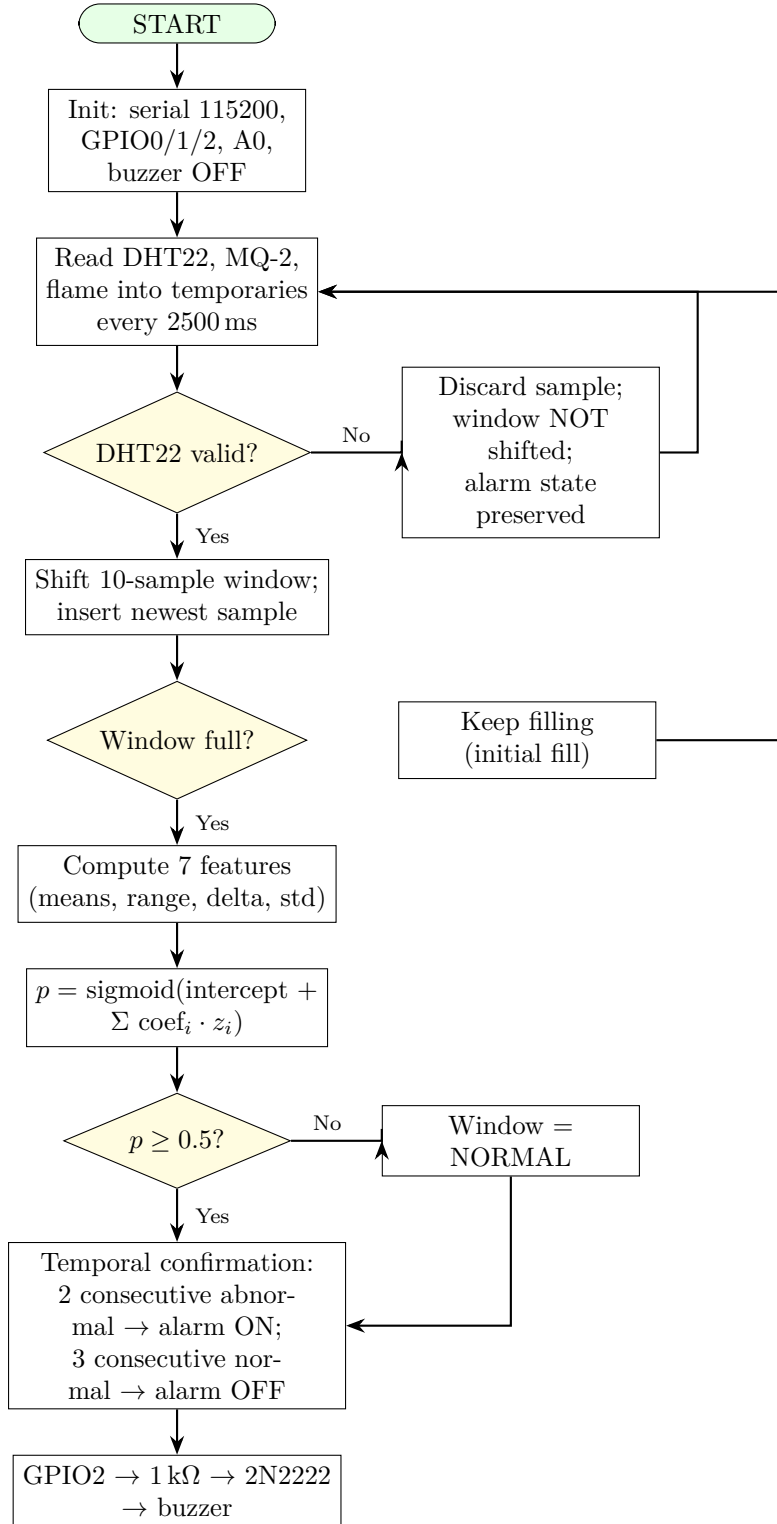


Figure 2: Firmware flowchart (one monitoring cycle; the cycle repeats from *Read sensors*).